Certificate-Gated Neural Proposers for Verifier-Grounded Action Scoring

PLACEHOLDER: Authors and affiliations

Working draft — ideal-paper roadmap

Abstract. When an exact verifier is available, the right architecture for action scoring is not an end-to-end neural policy. We study a hybrid pipeline in which a deterministic certificate-checked gate prunes structurally impossible candidates before a small learned scorer ranks the residual. On a prime-gap benchmark with residue-sieve features removed from the scorer input, the raw scorer reaches 9.13% top-1; a fixed hand-wheel gate raises this to 45.12%, and a learned rule gate reaches 68.31% at 100% top-10 with an ≈4× reduction in model latency. The symbolic gate itself is produced by structural rule search against a soundness-certificate verifier: rule search proposes 32,637 candidates, certificate-verifies 4,692, and accepts 52 rules with zero false rejections, recovering 79.64% of wheel-equivalent verifier work. A fixed 16-rule budget baseline shows the current neural proposer is not essential: grammar order and a simple structural heuristic save 73.22% of exact work versus 67.03% for the neural proposer. Under sparse-exact-reward policy gradient on the same task, strict verifier RL inflates logit scale (ECE 0.070 → 0.127) without improving ranking; a KL-toward-prior + entropy regularizer keeps ECE at 0.060 at the same top-k. Prime gaps are used as a substrate, not the goal; we argue the protocol is transferable to any exact-verifier domain.

1. Introduction

Exact verifiers are common in domains where action correctness can be checked algorithmically even when discovery cannot: theorem proving, program synthesis, combinatorial search, formal shortest-path problems. The dominant style in modern learned policy work treats the verifier as a reward signal for an end-to-end neural policy. We argue this is the wrong default when the candidate space contains many structurally impossible actions that the verifier can cheaply eliminate.

Our thesis is the **certificate-gating loop**: a learned scorer should consume the **survivors** of a verifier-checked symbolic filter, and the filter itself can be produced by a rule proposer gated by a soundness-certificate verifier. The scorer ranks the residual; the gate eliminates the rest; nothing the gate emits can violate correctness because every rule it accepts carries a deterministic proof.

We make three claims, each tied to a concrete experiment in the released artifact:

(C1) **Certificate-gating dominates raw scoring.** On the canonical prime-gap benchmark, the minimal-feature raw scorer reaches 9.13% top-1. The same scorer reaches 45.12% under a fixed hand-wheel gate and **68.31% top-1 under a learned rule gate** at 100% top-10. The dominating effect is the gate, not the depth of the scorer.

(C2) **Certificate-gated rule search discovers sound symbolic gates.** A structural neural proposer over a small rule grammar, paired with a soundness-certificate verifier, proposes 32,637 rules, certificate-verifies 4,692, and produces 52 accepted rules with zero false rejections, recovering 79.64% of wheel-equivalent verifier work. The soundness guarantee is a property of the certificate, not of training; fixed-budget baselines show the current neural proposer does not beat grammar order.

(C3) **KL-regularized verifier RL prevents calibration collapse.** Strict policy gradient on sparse exact rewards inflates logit scale rather than improving ranking; a standard KL-to-prior + entropy term restores calibration at unchanged top-k.

We use prime gaps as the experiment domain because they offer a fast deterministic verifier at all scales, a compact totally-ordered candidate space, an exhausted classical literature, and rich algebraic structure for the rule grammar. **We are not claiming improved prime generation.**

2. Related Work

PLACEHOLDER — Required citations and 1-paragraph positioning against each:

- Process supervision (Lightman et al., 2023): we use a similar verifier-as-step-checker pattern; their reward model is learned, ours is exact-by-construction.
- Verifier-guided decoding / best-of-N (Cobbe et al., 2021; Hosseini et al., 2024): we gate **before** scoring, not after.
- AlphaProof / AlphaGeometry symbolic-neural search (Trinh et al., 2024): our rule synthesis is structurally similar but verifier-grounded by certificate rather than by proof completion.
- RLHF KL-regularized policy gradient (Stiennon et al., 2020; Ouyang et al., 2022): our Finding 3 is this objective applied to exact-verifier rewards.
- Neural-symbolic program synthesis (NeSy literature).

3. Method

3.1. The Certificate-Gating Loop

The reusable architecture is:

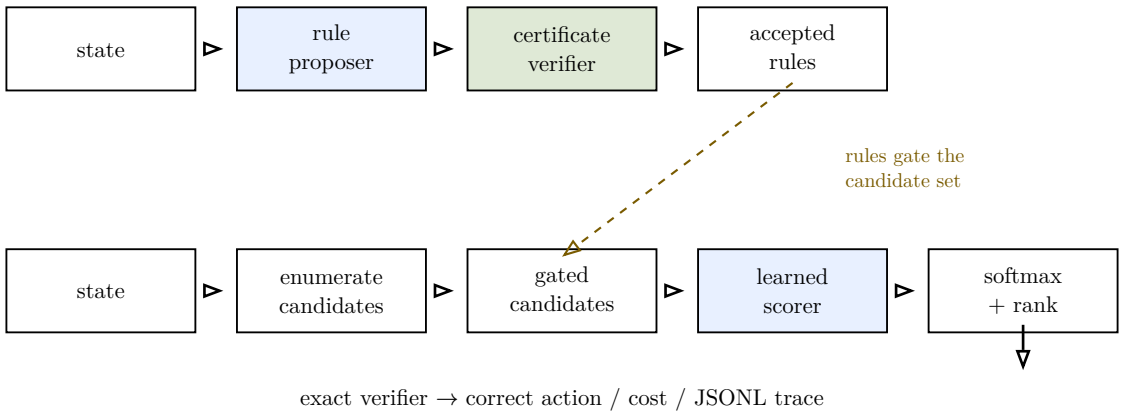


Figure 1: The certificate-gating loop. A rule proposer (top) produces candidate rules that a soundness-certificate verifier accepts or rejects. Accepted rules deterministically prune the candidate set; a shared neural scorer ranks the survivors; the exact verifier supplies the correctness signal.

The design invariant is **the model proposes, the verifier decides, the trace remembers.** A rule r is accepted only if a deterministic algebraic check certifies that r cannot reject the true action on any state in the support set, so soundness does not depend on the proposer being well-trained.

3.2. Trace and Checkpoint Contract

Every step writes a JSONL row with schema `crlp.trace.v1`, including a stable per-action identifier, the candidate scores and probabilities, the selected and target indices, the verifier result, and verifier cost. Replay resolves `target_index` → stable `action_id` → regenerated candidate row, so reordering action ids cannot silently change replay semantics. Checkpoints carry SHA-256 model

identity, a `parent_model_hash`, a `feature_encoder_version`, and a `feature_set`; comparison reports (`crlp.compare.v1`) carry both hashes, the evaluated `feature_set`, and machine-readable reliability bins.

4. Experiment Domain: Prime Gaps

4.1. Setup

The state is a prime p; the candidate space is gaps $g \in \{2, 4, ..., 256\}$ plus an overflow action; the verifier returns the true next-prime gap. The default candidate encoding is the minimal feature set $[\log p, \text{norm_gap}, \text{overflow_flag}]$. A full-sieve feature set that adds precomputed divisibility flags for $p + g$ modulo small primes is retained only as a leakage ablation. The scorer is a shared MLP returning one scalar per candidate; softmax over candidates produces the proposal distribution.

The default split is $p < 800,000$ train, $800,000 \leq p < 1,000,000$ validation, $1,000,000 \leq p \leq 3,000,000$ test; all reported numbers use 30,000 held-out test states unless stated otherwise.

4.2. Why this domain

- A fast deterministic verifier at all scales (segmented sieves, wheel filters, block proofs).
- A compact totally-ordered candidate space.
- An exhausted classical literature — we cannot beat a sieve, which sharpens the question to "what does the learned component contribute under perfect feedback?"
- Rich algebraic structure for the rule grammar (residue classes, divisibility certificates).

5. Finding 1: Certificate-gating dominates raw scoring

The same trained scorer is evaluated under three gate conditions on the canonical 30,000-sample held-out benchmark. Top-1 results:

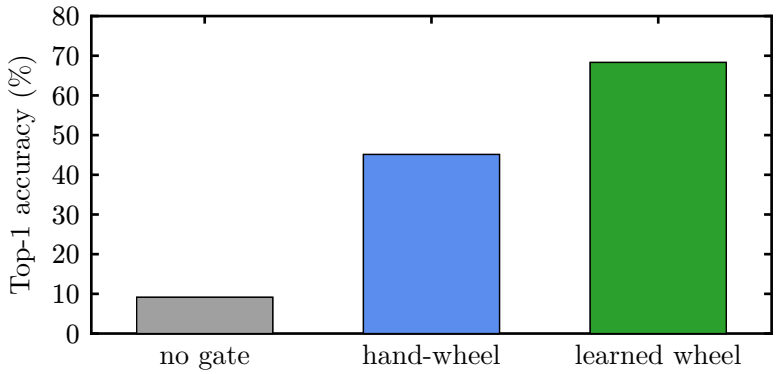


Figure 2: Top-1 accuracy by gate on the canonical 30,000-sample held-out benchmark. The clean raw-to-learned delta is **9.13 → 68.31**. The fixed-hand-wheel-to-learned delta is **45.12 → 68.31** (+23.2pp).

The former 45.12% raw-scorer result is now a full-sieve-feature leakage ablation, not the headline.

Full metrics:

Gate	Top-1	Top-10	NLL	ECE	ms/sample
no gate	9.13%	78.01%	2.9994	0.1076	0.0345
hand-wheel	45.12%	99.77%	1.5902	0.0834	0.0356
learned wheel	**68.31%**	**100.00%**	**1.0172**	0.1379	**0.0096**

Table 1: Gate ablation, minimal-feature scorer, identical weights across rows.

PLACEHOLDER — Required to elevate C1 from "load-bearing on prime-gap" to "general":

- **E1.1** — Random-rule gate at matched verifier-call budget: replace the learned rule set with k random sound rules and report the top-1 / top-10 / NLL curve as k varies. Resolves whether the **content** of the learned rules matters or only their count.
- **E1.2** — Extend the second-domain `grid_path` check beyond the current budgeted legality gate to admissible-heuristic pruning. Same scorer + three gates + same table.
- **E1.3** — Investigate the learned-wheel calibration tradeoff. The learned wheel improves ranking sharply but has higher ECE than raw minimal and hand-wheel rows in the default run.
- **E1.4** — Confidence intervals across ≥ 5 seeds for the 45.12 → 68.31 delta.

6. Finding 2: Certificate-gated rule search discovers sound symbolic gates

The canonical rule grammar is `reject candidate n when n > m and n % m == r`; the certificate verifier accepts a rule iff $\gcd(m, r) > 1$ (so n is necessarily composite). The expanded grammar can also express conditional gap filters, but that mode is not the canonical result. Zero false rejections is a structural property of the certificate. The proposer input is structural-only: condition kind, moduli, residues, zero-residue indicators, gcd ratios, and derived candidate residue. Empirical support, utility, and false-rejection counts are labels or diagnostics, not proposer features.

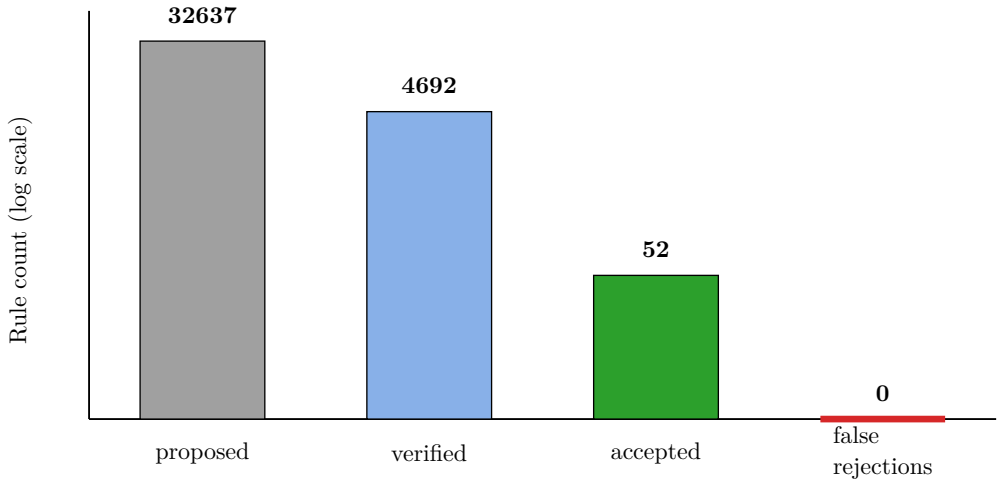


Figure 3: Rule-search acceptance funnel. Of 32,637 proposed rules, 4,692 pass the soundness certificate, 52 survive support and false-rejection screens, and **zero** falsely reject the true next-prime gap. The accepted set recovers 79.64% of wheel-equivalent verifier work.

Rule ordering	Budget	Exact probes	Saved
neural structural	16	2.37	67.03%
random, seed 20240511	16	3.86	46.30%
grammar order	16	1.93	73.22%
structural heuristic	16	1.93	73.22%
validation-count oracle	16	1.93	73.22%

Table 2: Matched 16-rule-budget proposer baselines over the same grammar, split, and certificate verifier. Lower exact probes is better.

What this evidence supports. "Certificate-gated structural rule discovery over this grammar produces a sound, useful rule set without support-count feature leakage." The soundness comes from the certificate; the coverage comes from the grammar plus accepted certified filters.

What this evidence does not support. "The neural proposer, specifically, is essential." Table 2 is a negative result for that stronger claim: simple deterministic orderings beat the neural structural proposer at the fixed 16-rule budget. The next neurosymbolic experiment must use a richer grammar and show that learned ordering beats grammar-order and structural-heuristic baselines under the same budget.

PLACEHOLDER — Required to elevate C2 from "the verifier is doing useful work" to "the learned proposer is useful":

- **E2.1** — Repeat the fixed-budget proposer baseline across random seeds and rule budgets, reporting confidence intervals.
- **E2.2** — Generalize the grammar: extend to conjunctive rules (`if p % m == a and ... then reject ...`), report whether the proposer scales.
- **E2.3** — Show one **worked example** of an accepted rule the hand-wheel does not capture and explain why it generalizes.

7. Finding 3: KL-regularized verifier RL prevents calibration collapse

Starting from a single supervised checkpoint, we clone two RL branches: strict policy gradient on exact rewards, and the same gradient regularized with KL-to-supervised-prior + entropy.

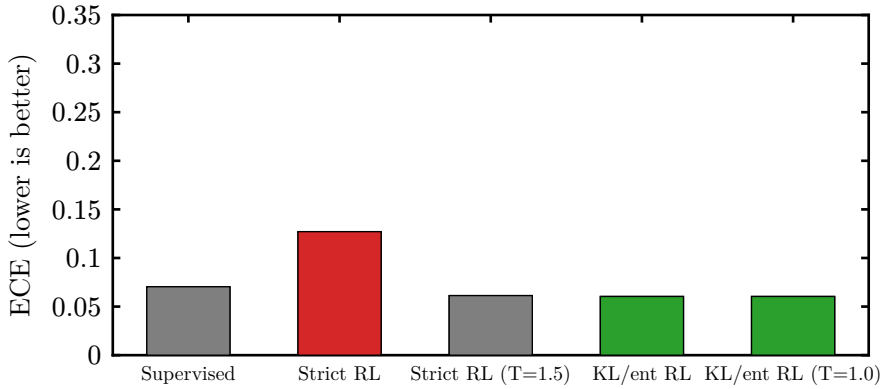


Figure 4: Expected calibration error by RL variant. Strict verifier RL degrades calibration (ECE 0.070 → 0.127) while leaving top-1 **unchanged at 9.13%** across all variants. Post-hoc temperature scaling needs $T = 1.5$ to recover; KL/entropy regularization fixes it at training time without that crutch.

Policy	T	Top-1	Top-10	NLL	ECE
Supervised	1.00	9.13%	78.01%	2.9170	0.0704
Strict verifier RL	1.00	9.13%	78.01%	3.0611	**0.1271**
Strict RL, temp-scaled	1.50	9.13%	78.01%	2.9063	0.0613
KL/entropy RL	1.00	9.13%	78.01%	2.9056	0.0604
KL/entropy RL, temp-scaled	1.00	9.13%	78.01%	2.9056	0.0604

Table 3: RL ablation, single cloned supervised checkpoint, default split.

The mechanism: strict policy gradient on a sparse exact reward signal pushes logit magnitudes without changing their relative ordering. The KL penalty bounds movement away from the reference policy;

the entropy bonus discourages collapse to a single overconfident action. Together they reproduce the standard RLHF training stabilization, applied here to verifier rewards instead of preference rewards.

7.1. Reliability Diagram

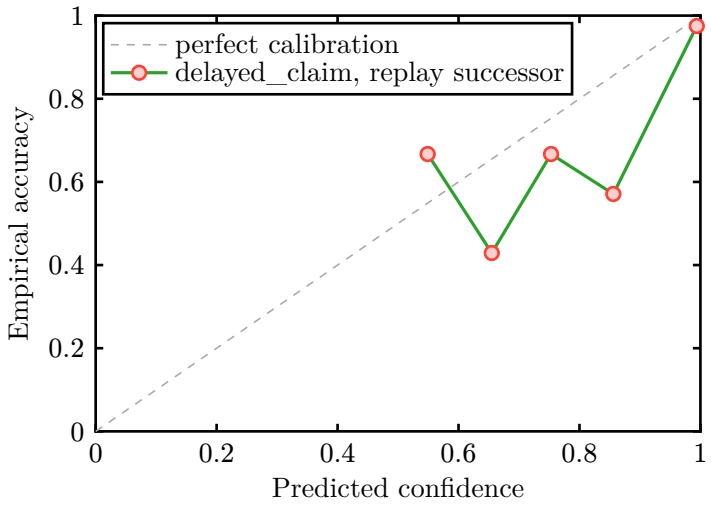


Figure 5: Reliability diagram for the trace-replay successor on `delayed_claim`. Bins from `results/latest/comparison.jsonl`. The high-confidence bin (894 of 1000 samples, $\hat{p} = 0.994$, $\hat{p}_{\text{emp}} = 0.975$) carries most of the mass; the low-population mid bins are noisier.

PLACEHOLDER: Replace with the prime-gap reliability diagram for supervised, strict-RL, and KL-RL once `make prime-gap` is run with `reliability_bins` capture enabled.

PLACEHOLDER — Required to elevate C3 from "true on prime-gap" to "a property of sparse-exact-reward RL":

- **E3.1** — Run strict-vs-regularized RL on `grid_path` and `delayed_claim` with the same protocol; report the analogous Table 3 for each.
- **E3.2** — Sweep the KL coefficient and entropy weight on a coarse grid for each task, plot the calibration-ranking Pareto frontier.
- **E3.3** — Per-bin reliability diagrams for **strict-RL vs KL-RL on prime-gap** (the current diagram uses delayed-claim as a stand-in because that's the only task whose `reliability_bins` are currently captured).
- **E3.4** — Confidence intervals across ≥ 5 seeds for the strict-vs-KL ECE delta.

8. Substrate

The hybrid architecture is the contribution; the substrate is the discipline that makes the claims auditable. Three pieces matter and would be a brief methods section in a longer paper:

- **Stable action IDs.** Trace rows carry `schema: "crlp.trace.v1"`, `trace_version: 1`, and `action_id_contract: "stable_unique_action_ids"`. Replay resolves `target_index` to the trace row's `action_id`, then maps that stable id back into the regenerated candidate row.
- **Checkpoint identity.** SHA-256 over the exact checkpoint bytes, plus `parent_model_hash`, `feature_encoder_version`, and `feature_set` in metadata. Successor models declare their parent; comparison reports carry both hashes and the evaluated `feature_set`.
- **Pure C, no third-party deps.** The neural library is dense layers, sigmoid/ReLU/tanh/linear, MSE and softmax cross-entropy, SGD, batched 4-by-4 tiled forward/backward, cloned networks, action scoring, pairwise ranking loss, and cached policy-gradient updates with optional KL/entropy

regularization. ≈3k lines for the core; the prime-gap task is split into focused C modules for data, features, rule search, training, evaluation, and replay.

9. Limitations

L1. **Proposer learning not solved (Finding 2).** The fixed-budget baseline shows grammar order and a simple structural heuristic beat the current neural proposer. The load-bearing claim is certificate-gated rule discovery, not learned proposer superiority.

L2. **Cross-task verifier-RL comparison missing (Finding 3).** The pipeline supports it; the comparison is not in canonical results yet.

L3. **Limited cross-domain gate coverage (Finding 1).** `grid_path` now has a budgeted legality-gate sanity check, but the headline hybrid-wins table is still prime-gap only.

L4. **Statistical rigor.** Default `make prime-gap-experiments` runs 1 seed; the harness supports more. Deltas need confidence intervals across ≥ 5 seeds.

L5. **Learned-wheel calibration tradeoff** needs a reliability-diagram pass.

L6. **No literature comparison** on the calibration-collapse mechanism yet.

10. Conclusion

> **PLACEHOLDER —** Write the conclusion last. Three sentences max: (a) certificate-gating is the architectural claim, (b) verifier-grounded rule synthesis is the constructive claim, (c) KL-regularized verifier RL is the training-stability claim. Avoid claiming the protocol scales; that is future work, not a result.

11. Roadmap to the ideal paper

This appendix tracks every experiment the current draft asks for but does not yet have. The goal is that every **PLACEHOLDER: ...** marker in the body resolves to one of these rows.

ID	Experiment	Effort	Defends
E1.1	Random-rule gate at matched verifier-call budget	2d	C1
E1.2	Extend `grid_path` beyond legality gate with admissible-heuristic pruning	3-4d	C1
E1.3	Investigate learned-wheel calibration tradeoff	1d	C1
E1.4	CIs across ≥ 5 seeds for $45.12 \rightarrow 68.31$ delta	0.5d	C1
E2.1	Fixed-budget proposer baselines across random seeds and budgets	1d	C2
E2.2	Conjunctive rule grammar extension	3-5d	C2
E2.3	Worked example of an accepted rule the hand-wheel misses	0.5d	C2
E3.1	Strict-vs-regularized RL on `grid_path` and `delayed_claim`	2-3d	C3
E3.2	KL/entropy Pareto sweep per task	2d	C3
E3.3	Reliability diagrams for strict-RL and KL-RL on prime-gap	0.5d	C3
E3.4	CIs across ≥ 5 seeds for strict-vs-KL ECE delta	0.5d	C3
W1	Related-work positioning paragraphs (see §2)	1d	all
W2	Conclusion + abstract revision after experiments	0.5d	all

Table 4: Roadmap. Total estimated effort: ≈3 weeks of focused work to take the current draft from "defensible as an arxiv preprint" to "submission-ready with all headline claims independently supported."

Minimum viable submission. E2.1, E1.3, E3.3, W1, W2. ≈4 days. Leaves Findings 2 and 3 with qualified scope (prime-gap only) but every claim in the body has a defensible experiment behind it.

Strong submission. Add E1.2, E2.3, E3.1, E1.4, E3.4. ≈2 weeks. All three findings have either a cross-task replication or a baseline strong enough to rule out the obvious alternative explanation.

Full ideal paper. Everything above. ≈3 weeks. The grammar extension (E2.2) is the optional ambition; if conjunctive rules generalize, the paper grows a fourth finding.